

Criteo 광고 데이터 파이프라인

실시간 수집 → Medallion 아키텍처 → BI 시각화

Kafka

Spark

S3 Iceberg

Glue Catalog

Athena

Airflow

Superset

프로젝트 개요 및 목표

프로젝트 배경

도메인: 광고 성과 분석 (Criteo Attribution Dataset)
HuggingFace 오픈소스 — impression / click / conversion 3종 이벤트
원본 데이터는 한 행에 조인된 형태 → 실시간 분리 스트림으로 재설계

목표: 실제 광고 데이터 플랫폼의 수집 → 저장 → 집계 → 시각화
흐름을 처음부터 끝까지 직접 구현하며 강의에서 다루었던 기술 이해

기술 스택

수집	Criteo Dataset (HuggingFace Streaming)
메시지큐	Apache Kafka — 3토픽 × 3파티션
스트리밍	Spark Structured Streaming
저장소	AWS S3 (Parquet + Iceberg)
메타스토어	AWS Glue Data Catalog
쿼리	Amazon Athena (서버리스 SQL)
오케스트레이션	Apache Airflow 3.x
시각화	Apache Superset

광고 이벤트 데이터 이해

impression / click / conversion — 지연 도착과 Attribution Window

impression

광고 노출 이벤트
사용자 화면에 광고가 보임
cost(노출 비용) 발생

도착 시점: 즉시 발생

click

광고 클릭 이벤트
사용자가 광고를 클릭
impression 발생 후 수초~수일 뒤 도착

도착 시점: impression 후
수초 ~ 수일

conversion

전환 이벤트
사용자가 구매/가입 완료
impression 발생 후 수일~수십일 뒤 도착

도착 시점: impression 후
수일 ~ 수십일

핵심 문제 — 같은 이벤트인데 도착 시점이 다르다

같은 사용자의 **impression**은 지금 저장했는데, 그에 대한 **click**은 7일 뒤, **conversion**은 30일 뒤에 도착

→ 파이프라인이 매일 배치로 돌더라도 오늘 impression의 click/conversion을 오늘 알 수 없음

→ Silver 레이어에서 이 '지연 도착' 문제를 2-Stage MERGE로 해결

Attribution Window — '이 impression 덕분에'를 인정하는 시간 한도

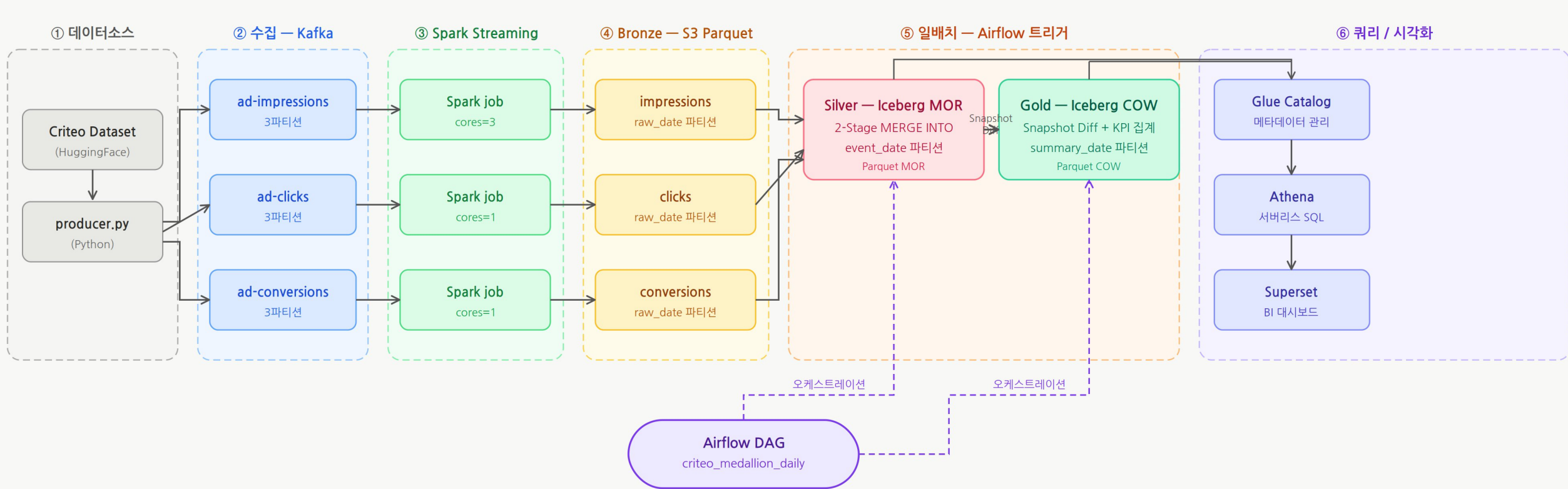
click window: 7일 — impression 후 7일 이내 click만 이 impression에 귀속 (업계 표준, Google·Meta 동일)

conversion window: 30일 — impression 후 30일 이내 conversion만 귀속

→ 이 window가 Silver 파티션 스캔 범위, Iceberg compaction 35일 기준, Gold KPI 정확도의 핵심 파라미터

전체 아키텍처

Criteo Dataset → Kafka → Spark → S3 Iceberg → Athena → Superset



왜 Apache Iceberg인가

일반 Parquet 대비 핵심 기능 비교

항목	일반 Parquet	Iceberg (이 프로젝트)
ACID 쓰기 (MERGE INTO)	직접 지원 없음 파티션 전체 DROP/INSERT 필요	✓ MERGE INTO 네이티브 지원 역등 upsert 가능
스냅샷 & Time Travel	불가 — 파일 덮어쓰면 이전 상태 복구 불가	✓ 스냅샷별 이전 상태 조회 Gold snapshot diff 핵심 기반
파티션 변경	파티션 컬럼 변경 시 파일 전체 재작성	✓ 메타데이터만 변경 데이터 파일 유지
파일 수준 통계	파티션 단위만 파티션 내 세밀한 pruning 불가	✓ 컬럼 min/max 통계 파티션 내 파일 pruning 가능
COW / MOR 전략	없음 — 항상 파일 전체 재작성	✓ 쓰기 패턴에 따라 선택 Silver=MOR, Gold=COW

Medallion 아키텍처 — 레이어별 설계 개요

Bronze	Silver	Gold
역할 원본 보존	역할 분석 가능 단위로 정제	역할 KPI 집계
포맷 S3 Parquet (append-only)	포맷 Iceberg MOR	포맷 Iceberg COW
파티션 키 raw_date / raw_hour (ingest_ts 기준)	파티션 키 event_date (impression 발생 기준)	파티션 키 summary_date (event_date와 동일)
처리 방식 Kafka 3토픽 → Spark Structured Streaming	처리 방식 중복 제거 + Attribution Join + MERGE INTO (일배치)	처리 방식 Silver → campaign×date 집계 MERGE INTO (일배치)
역등성 보장 없음 (원본 파일이 유일한 재처리 수단)	역등성 보장 MERGE ON (event_id, event_date)	역등성 보장 MERGE ON (summary_date, campaign)

Bronze — 실시간 수집 레이어

Kafka 3토픽 → Spark Structured Streaming job → S3 Parquet

Kafka 3토픽 × Spark Streaming job 구성

ad-impressions → Streaming job (cores.max=3)
impression만 cost 필드 있음 — 스키마 달라 토픽 분리 필수
ad-clicks → Streaming job (cores.max=1)
ad-conversions → Streaming job (cores.max=1)

토픽별 독립 Streaming job 컨테이너로 분리

→ impression 처리 지연이 click/conversion 수집에 영향 없음
→ Spark Worker 3대 × 3코어 = 9코어 중 Streaming 5코어 사용
→ 나머지 4코어는 Silver/Gold 일배치용

raw_date 파티션 = ingest_ts 기준 (수집 시각)

event_time(이벤트 발생 시각) 기준으로 하면?
오늘 수집된 conversion이 30일 전 파티션에 기록 → 파일 크기 예측 불가

ingest_ts 기준: 오늘 수집된 모든 이벤트가 오늘 파티션에 균등 적재

→ 장애 복구 시 '이 날짜 파티션부터 Silver 재처리' 범위 특정 가능
Silver에서 event_date(impression 발생 기준)로 재파티셔닝
→ Bronze는 append-only 원본 저장이므로 Iceberg 불필요

Spark Checkpoint — Streaming 장애 복구

S3에 토픽별 독립 checkpoint 저장

s3a:///.../checkpoints/kafka_to_raw/impressions/
s3a:///.../checkpoints/kafka_to_raw/clicks/
s3a:///.../checkpoints/kafka_to_raw/conversions/

Streaming job 크래시 후 재시작 시

→ 마지막 처리된 Kafka offset에서 이어서 처리
→ 중복/누락 없이 재개 가능

checkpoint 경로를 토픽별로 분리하는 이유

공유하면 offset 상태가 섞여 재처리 시 누락 또는 중복 발생

⚠ 한계

- Kafka 단일 브로커 (replication factor=1) — 브로커 장애 시 메시지 손실
→ 운영 환경에선 3+ 브로커 클러스터 + replication 구성 필요
- Spark Worker가 단일 머신에 모두 있어 실제 분산 환경 아님 (홍내 수준)
- producer.py 재시작 시 전체 데이터 처음부터 재발행 — 모든 이벤트 중복 가능 (Silver MERGE에서 흡수)

Silver — Attribution 2-Stage 일배치

Dedup + Attribution Join + Iceberg MERGE INTO (MOR)

왜 2-Stage인가 — 이벤트 도착 시점 문제

어제(6/24) Bronze raw_date=6/24에 들어오는 것:

impression: 어제 발생 → event_date = 6/24

click: 어제 도착, 원본 impression은 6/17~6/24 (click window 7일)

conversion: 어제 도착, 원본 impression은 5/25~6/24 (window 30일)

Impression 이벤트 발생 시간 기준 재파티셔닝

Stage 1: 어제 Bronze의 impression → Silver INSERT

event_date = impression 발생 시각 기준 (여러 파티션 분산 가능)

이미 Silver에 있는 impression은 INSERT 안 함 (먹등)

click=0, conversion=0 초기값으로 적재

Stage 2: 어제 Bronze의 click/conversion → Silver 7일/30일 파티션에서

eid로 impression 찾아 MERGE UPDATE

→ Silver가 중간 저장소 역할, 30일치 Bronze 스캔 불필요

왜 MOR(Merge-On-Read)인가

UPDATE 대상(click/conversion 매칭 impression): 전체의 ~2%

COW: 2% 행이 바뀌어도 그 파티션 파일 전체 재작성

MOR: delta file만 append → 쓰기 비용 절감

읽기 시 delta 병합 오버헤드 → 유지보수 DAG의 daily compaction이 해결

△ 백필 시 Silver도 명시 재처리 모드(--run-date-start 지정) 사용 필요
일배치 기본 모드(snapshot diff)는 같은 날 여러 번 실행 시 변경 감지가 깨짐

Gold — Snapshot Diff + KPI 집계

Silver 변경분만 재집계 → Iceberg MERGE INTO (COW) → 일배치

Snapshot Diff — Silver 변경 파티션만 재집계

아이디어: 일배치 → 오늘 silver에서 바뀐 것만 재집계하면 충분
Silver 전체를 매번 재스캔하는 건 낭비
(attribution window 30일치만 봐도 되지만 그것도 매일 전부 재집계하면 비효율)

구현: Silver Iceberg 스냅샷으로 변경 파티션 감지

- ① 오늘 Silver 첫 번째 커밋의 parent = 배치 이전 상태
- ② 현재 스냅샷 vs parent 스냅샷을 event_date별 행수 비교
- ③ 달라진 event_date = 오늘 변경 파티션 → Gold 재집계 대상
(당일 신규 + 지연 attribution으로 변경된 과거 파티션 자동 포함)

왜 COW(Copy-On-Write)인가

Gold는 Superset/Athena에서 가장 많이 읽히는 테이블 → 읽기 성능이 최우선
COW: 파티션 파일 재작성 후 항상 clean → Athena 스캔 성능 일정

Gold KPI — campaign_summary 스키마

impressions	COUNT(*)
clicks	SUM(click)
conversions	SUM(conversion)
unique_users	COUNT(DISTINCT uid)
CTR	clicks / impressions × 100
CVR	conversions / clicks × 100 (clicks=0 → NULL)
CPC	total_cost / clicks (clicks=0 → NULL)
CPA	total_cost / conversions (conversions=0 → NULL)
CPM	total_cost / impressions × 1,000
frequency	impressions / unique_users
avg_conv_delay	AVG(conversion_delay_sec ≥ 0) — sentinel(-1) 제외
CTC / VTC	click=1&conversion=1 / click=0&conversion=1

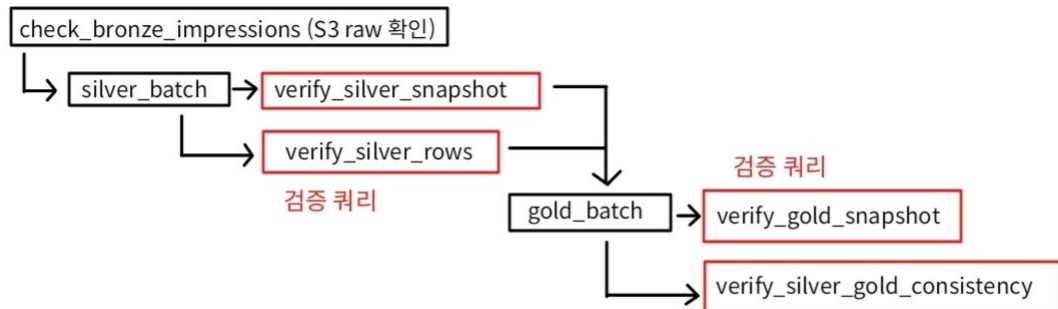
⚠ 백필 시 Gold는 명시 재처리 모드(--run-date-start 지정) 사용 필요

운영 자동화 — Airflow DAG 구조

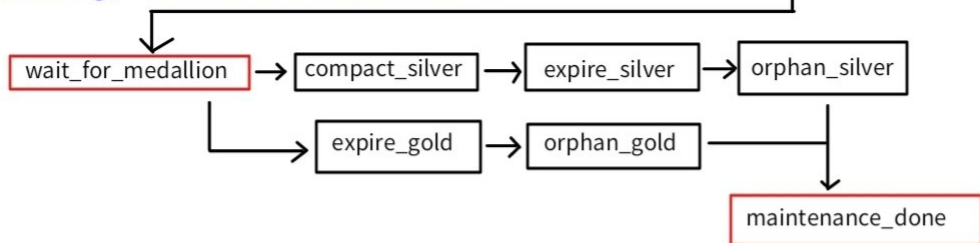
3개 DAG + 검증 게이트 + ExternalTaskSensor 의존성

DAG 의존 관계

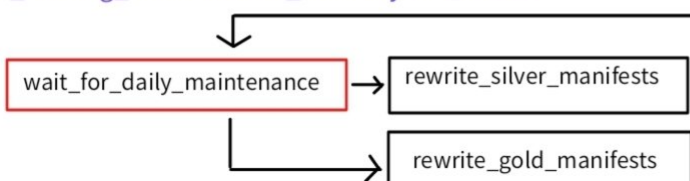
criteo_medallion_daily (매일 UTC 02:00)



criteo_iceberg_maintenance (medallion 완료 후)



criteo_iceberg_maintenance_monthly (매월 1일만)



⚠ 한계

- 검증 범위: 어제 파티션 1개만 확인
Attribution으로 갱신된 어제 이전 파티션 Silver ↔ Gold 정합성 미커버
- 이 DAG 구조는 Silver/Gold 테이블이 각 1개라는 전제에 의존
새 테이블 추가 시 전체 DAG가 무너지는 구조 → 레이어별 유지보수 DAG 분리가 더 적합한 설계였음
- 백필: --run-date-start/end 인자 있으나 Airflow UI에서 날짜 지정 연결 미구현
— docker compose 명령 직접 실행 필요

Iceberg 유지보수 전략

Compaction / Expire / Orphan / Manifest 재편성 / metadata.json 보존

Silver (MOR) 일간 유지보수

compact_silver (rewrite_data_files)

MOR MERGE가 매일 delete file 누적 → base + delta 병합 오버헤드 제거

expire_silver_snapshots

스냅샷 정리

remove_silver_orphans

고아파일 정리

metadata.json 보존 설정 (TBLPROPERTIES)

커밋마다 새 metadata.json 생성 → 무한 누적 방지

Gold (COW) 일간 유지보수

Compaction: 현재 규모에서 불필요

COW는 delete file 없음 → MOR처럼 매일 필수 아님

현재 데이터 규모: 파티션당 Parquet 파일 1개 수준 → 소형 파일 누적 없음
데이터가 커지면 소형 파일 병합을 위한 compaction 필요해질 수 있음

expire_gold_snapshots

스냅샷 정리

remove_gold_orphans

고아파일 정리

metadata.json 보존 설정

월간 유지보수 (매월 1일)

rewrite_manifests (Silver + Gold 병렬)

스냅샷 누적으로 manifest 파편화 → 소수의 큰 파일로 재편성
데이터 파일 변경 없이 쿼리 플래닝 오버헤드 감소
현재 규모(일 파티션 1개)에서는 월 1회로 충분

MERGE와 Compaction 충돌 방지

ExternalTaskSensor: medallion DAG gold_batch SUCCESS 감지 후 maintenance 시작

→ Silver/Gold MERGE 완전히 끝난 뒤 Compaction 실행 → Iceberg 낙관적 잠금 충돌 없음

Superset 대시보드

Gold → 비즈니스 KPI | Silver → 운영 모니터링

비즈니스 대시보드 — Gold 기반 (Campaign팀/BI팀)

데이터 소스: Gold campaign_summary
(집계 테이블 → Athena 스캔 비용 낮음)

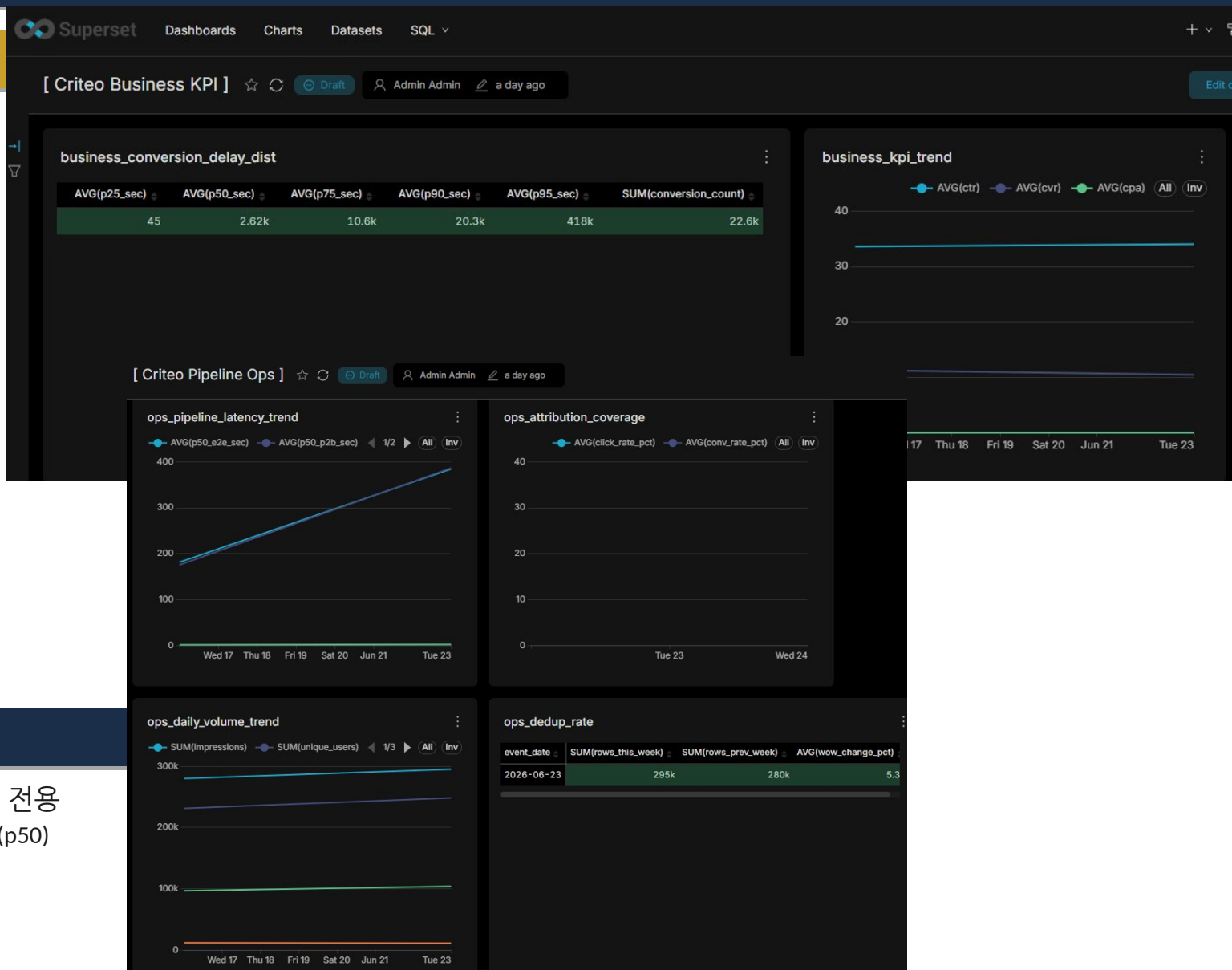
business/01 30일 캠페인별 KPI 추이
CTR / CVR / CPC / CPA / CPM / frequency
click-through conversion vs view-through conversion

business/02 Conversion 지연 분포 (주간 refresh)
p25 / p50 / p75 / p90 / p95
Attribution window(30일) 적정성 검증 지표

Superset Dataset 캐싱 (미구현 — 적용 시 비용 절감 가능)
24h 캐싱 설정 시 하루 1번 Athena 스캔으로 여러 팀 조회 커버

운영 대시보드 — Silver 기반 (DE용)

Silver를 데이터 소스로 사용 — 파이프라인 정상 동작 여부 확인 전용
일별 볼륨 추이 / attribution 커버리지 / 단계별 파이프라인 지연 추이 (p50)
운영 이상 신호(급증/급감/attribution 0%)를 시각적으로 모니터링



설계 한계 — 고려하지 못한 것들

인프라 HA 미고려

Kafka 단일 브로커 — 장애 시 메시지 손실
Spark Worker가 단일 물리 머신 — 실제 분산 아님
장애 복구 절차 미설계

플랫폼 확장성

Silver/Gold 테이블이 여러 개 생기는 상황을 생각 못 했음
지금 구조는 테이블 1:1 하드코딩 — 테이블 늘면 DAG 전체 수정
레이어별 유지보수 DAG 분리가 더 적합했을 것
Bronze도 Silver 테이블 늘면 읽히는 빈도 증가
→ Bronze compaction Spark job 별도 필요해질 수 있음

스케일아웃 전략 없음

데이터 10x 증가 시 파티션 전략 / 코어 배분 어떻게 바뀌어야 하는지 고려 못 했음

날짜 기준 UTC 고정 (UTC vs KST)

Kafka·Spark·Airflow·S3 모두 UTC 기준으로 동작→ 날짜 경계 처리 주의
현재 Airflow를 KST 낮 시간대에 실행해 회피 중 (근본 해결 아님)

백필 운영 절차 미흡

CLI 인자는 있는데 Airflow UI에서 날짜 지정하는 연결 미구현

Compaction 정렬 전략 미고려

현재 bin-pack: 파일 크기만 맞춤, 정렬 없음
sort/z-order 적용 시 attribution JOIN·Superset 쿼리 성능 개선 가능

Gold 집계 버그 — 발표 준비 중 발견 (미수정)

Silver 스냅샷 비교로 변경 파티션만 Gold 재집계하는 방식인데
click·conversion은 기존 행 값만 바뀌어 행 수 그대로 → 감지 누락

프로젝트 진행 후기

CS 기반 부족 — 환경 설정에서 막혔음

환경변수·포트·네트워크·엔드포인트 등 처음 들어보는 개념들이 많아 이해 어려움
.config() 자격증명·docker-compose·Dockerfile 설정은
Claude Code에 맡기고 '앱이 돌아가면 된 거겠지'로 진행

기능 단위 테스트를 안 했음

기능 여러 개 붙이고 파이프라인 한번에 돌려서 오류 역추적
→ 기능 하나 붙일 때마다 소규모 테스트했으면 더 효율적

적재 데이터 직접 검증 안 했음

파이프라인이 오류 없이 돌면 데이터가 맞겠지 하고 넘어감
→ 테이블 만들 때마다 소규모 Athena 쿼리로 여러 케이스 확인했어야 함

Spark 자원 배분 고민 부족

worker-executor 조정으로 처리량 늘린다는 건 알지만
job별 구체적 자원 설계는 못 해봄

Claude Code 처음 써봤는데 AI가 제안한 코드 읽기부터 막히는 문제

구현 됐다고 하면 코드 안 읽고 패스하는 식으로 진행
코드 읽기부터 막히는 수준인데 직접 쓰기까지는 갭이 얼마나 클지
바이브코딩·노코드 추세인데 코드 읽기/쓰기에 대한 공부가
어느 정도로 필요할지 방향이 안 잡힘

감사합니다

Kafka → Spark Streaming → S3 Iceberg (MOR / COW) → Athena → Superset
Attribution Window | Snapshot Diff | MERGE INTO 먹등성 | Airflow 자동화